

04 Random Forest

EP

2026-07-19

Random Forest: Regression

```
library(targets)
library(tidymodels)
library(ranger)
library(sf)
library(dplyr)
library(ggplot2)
library(corr)

set.seed(399)
```

Load Data

Read in the final grid model and transform the simpson scores. This isn't strictly necessary for Random Forest but allows for direct comparison with the beta regression model.

```
# Uncomment if using pipeline data:

#setwd("~/CSCT_Pipeline")

#grid_model_all <- tar_read(grid_model_all)

#df <- grid_model_all %>%
  #sf::st_drop_geometry()
```

```
#If running with downloaded grid data use:

grid_model_all <- st_read("Data/grid_model.gpkg", quiet = TRUE)

df <- grid_model_all %>%
  sf::st_drop_geometry()
```

```
#Adjust simpson scores and set predictors
n <- nrow(df)
df$simpson_adj <- (df$simpson * (n - 1) + 0.5) / n

predictors <- c("n_days", "dtm_elev", "avg_imperv", "rofsw_risk", "rofrs_risk",
```

```
"avg_area_ha", "treeht", "weighted_precip", "weighted_temp",
"dist_to_river", "pct_greenpace")
```

Spatial cross-validation setup

Using distinct cities could introduce some spatial autocorrelation between cities but also between neighbouring grid squares that are likely to be similar naturally. This method of cross-validation ensures all rows from the same city remain in either the training or test set for each given fold. In this case, there are 5 cities so 5 folds are produced. Each fold contains 4 cities as training and one as the test set.

```
folds <- group_vfold_cv(df, group = city)

fold_city_lookup <- folds %>%
  mutate(city = purrr::map_chr(
    splits, ~ as.character(unique(assessment(.x)$city)))) %>%
  select(id, city)
```

Richness model

Model specification and tuning grid

This section defines the type of model (`rf_spec`) with number of trees fixed at 500. `richness_recipe` describes the data preprocessing and builds the model formula. `richness_wf` then bundles these together so future tuning can be run in a single call. `rf_grid` specifies the hyperparameter combinations to try. Here I have used 5 evenly spaced number of predictors between 2 and 8, and 5 evenly spaced values between 2 and 25 for the minimum number of observations at each terminal node.

```
# Specification
rf_spec <- rand_forest(
  mtry = tune(),
  min_n = tune(),
  trees = 500) %>%
  set_engine("ranger",
    importance = "permutation",
    respect.unordered.factors = "order") %>%
  set_mode("regression")

# Model formula
richness_recipe <- recipe(
  as.formula(paste("sp_richness ~", paste(predictors, collapse = " + "))),
  data = df) %>%
  step_naomit(all_predictors(), all_outcomes())

#Workflow: combine preprocessing recipe and RF specification
richness_wf <- workflow() %>%
  add_recipe(richness_recipe) %>%
  add_model(rf_spec)

#Construct hyperparameter grid
rf_grid <- grid_regular(
```

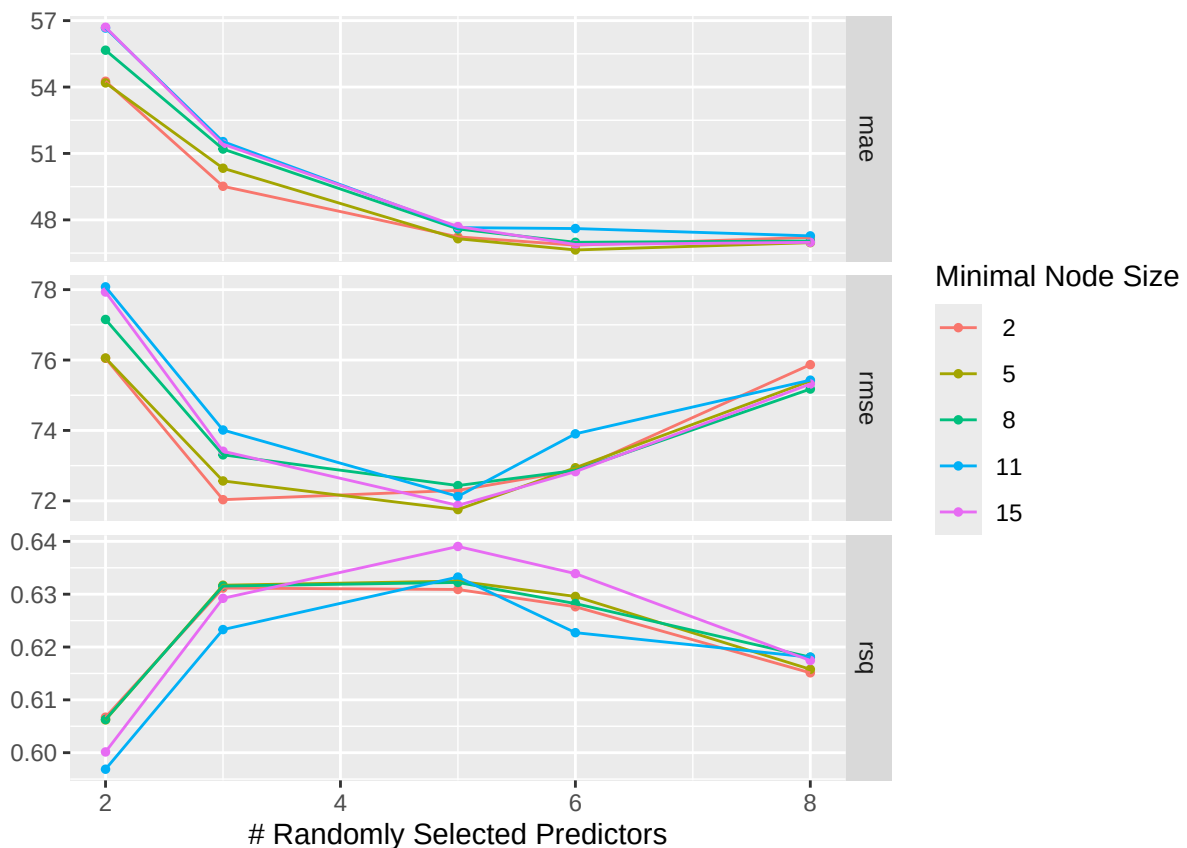
```
mtry(range = c(2, 8)),
min_n(range = c(2, 15)),
levels = 5) #Total of 25 combinations in the grid.
```

Tune via leave-one-city-out cross-validation

This runs the 25 hyperparameter combinations on each of the 5 folds, a total of 125 models are fit. For each, three performance metrics are calculated: Root mean square error (RSME), R^2 , and Mean Absolute Error (MAE). Performance (using RSME) is plotted to visually demonstrate which hyperparameters perform best, and top 5 combinations are printed as a table.

```
richness_res <- tune_grid(
  richness_wf,
  resamples = folds,
  grid      = rf_grid,
  metrics   = metric_set(rmse, rsq, mae))

autoplot(richness_res)
```



```
show_best(richness_res, metric = "rmse", n = 5) %>%
  select(mtry, min_n, mean, std_err) %>%
  mutate(across(c(mean, std_err), ~ round(.x, 3))) %>%
  rename(
    "mtry" = mtry,
```

```

  "min_n" = min_n,
  "Mean RMSE" = mean,
  "Std. error" = std_err
) %>%
knitr::kable(caption = "Top 5 hyperparameter combinations:
  species richness (ranked by RMSE)")

```

Table 1: Top 5 hyperparameter combinations: species richness (ranked by RMSE)

mtry	min_n	Mean RMSE	Std. error
5	5	71.752	17.415
5	15	71.871	18.025
3	2	72.034	18.982
5	11	72.128	17.542
5	2	72.297	17.389

Finalize and fit best model

This selects the best performing model and uses the original `richness_wf` to plug in the best hyperparameters. The final model is then run on the entire dataset.

```

best_richness <- select_best(richness_res, metric = "rmse")
final_richness_wf <- finalize_workflow(richness_wf, best_richness)
richness_fit <- fit(final_richness_wf, data = df)

richness_engine_fit <- richness_fit %>% extract_fit_engine()

tibble::tibble(
  Parameter = c("mtry", "min.node.size", "num.trees",
    "OOB prediction error (MSE)", "R2 (OOB)"),
  Value = c(
    richness_engine_fit$mtry,
    richness_engine_fit$min.node.size,
    richness_engine_fit$num.trees,
    round(richness_engine_fit$prediction.error, 3),
    round(richness_engine_fit$r.squared, 3)
  )
) %>%
knitr::kable(caption = "Final random forest model: species richness")

```

Table 2: Final random forest model: species richness

Parameter	Value
mtry	5.000
min.node.size	5.000
num.trees	500.000
OOB prediction error (MSE)	7180.524
R ² (OOB)	0.606

The richness model shows an Out of Bag (OOB) $R^2 = 0.606$ and RSME = 71.752.

Cross-validated performance

Extract the performance metrics, by city, for the best performing model (i.e. the final model used).

```
richness_cv_metrics <- collect_metrics(richness_res, summarize = FALSE) %>%
  filter(mtry == best_richness$mtry, min_n == best_richness$min_n) %>%
  left_join(fold_city_lookup, by = "id")

richness_cv_metrics %>%
  select(city, .metric, .estimate) %>%
  tidyr::pivot_wider(names_from = .metric, values_from = .estimate) %>%
  mutate(across(where(is.numeric), ~ round(.x, 3))) %>%
  rename(
    "City (Held-Out)" = city,
    "RMSE" = rmse,
    "R2" = rsq,
    "MAE" = mae
  ) %>%
  knitr::kable(caption = "Cross-validated performance by city:
    species richness")
```

Table 3: Cross-validated performance by city: species richness

City (Held-Out)	RMSE	R ²	MAE
Nottingham	54.143	0.601	37.012
Exeter	71.231	0.608	51.391
Newcastle	50.186	0.744	32.648
Bristol	139.045	0.437	85.811
Gateshead	44.154	0.773	28.877

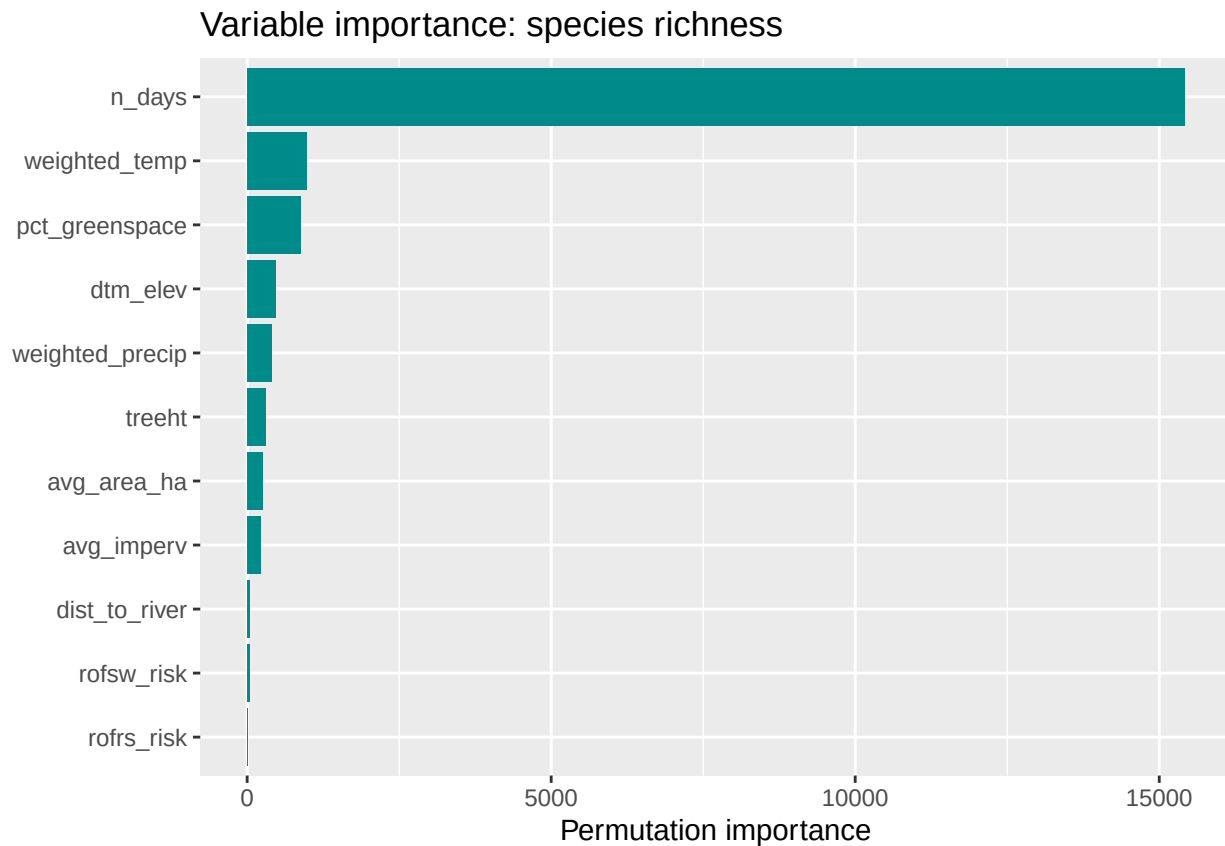
Spatial cross-validated performance by city identifies Bristol as an outlier, with a much larger RSME (139.045) and R^2 (0.437), suggesting that Bristol's grid make-up differs from the cities in some way. This could potentially be due to a different pool of species types or city environment (e.g. more precipitation or built up areas).

Variable importance

This plots the variable importance.

```
p1 <- richness_fit %>%
  extract_fit_engine() %>%
  ranger::importance() %>%
  sort(decreasing = TRUE) %>%
  as.data.frame() %>%
  tibble::rownames_to_column("variable") %>%
  rename(importance = 2) %>%
  ggplot(aes(x = reorder(variable, importance), y = importance)) +
  geom_col(fill = "cyan4") +
  coord_flip() +
  labs(title = "Variable importance: species richness",
    x = NULL,
```

```
y = "Permutation importance")
p1
```



```
#ggsave("VI_RF_Reg_Rich.png", p1, width = 12, height = 5, dpi = 300)
```

This shows `n_days` as the dominant predictor. This is understandable since a greater sampling effort will lead to more species records and, therefore, a better idea of the true richness. However, it is so dominant that it is worth checking for any correlation between predictors to ensure other environmental predictors aren't being masked.

Simpson Model

This process is then applied to the Simpson diversity scores.

Specify

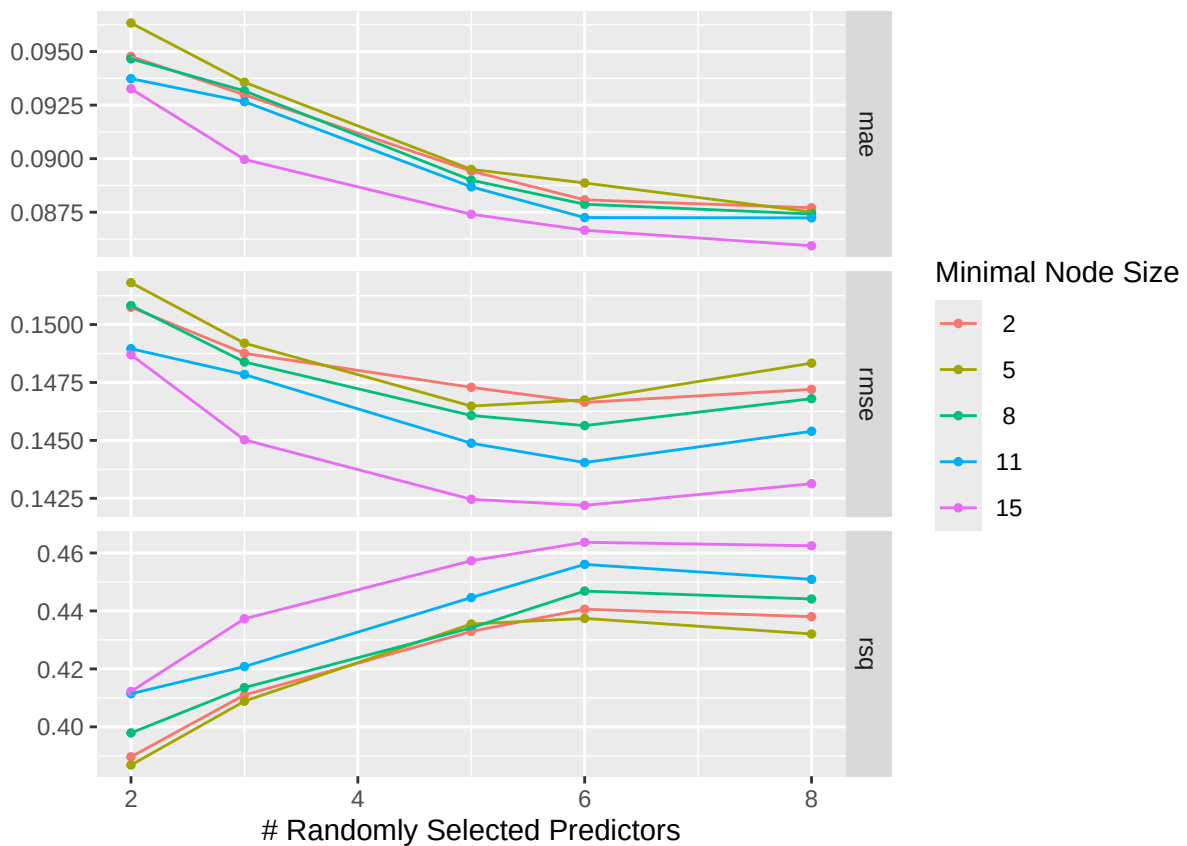
```
simpson_recipe <- recipe(
  as.formula(paste("simpson_adj ~", paste(predictors, collapse = " + "))),
  data = df) %>%
  step_naomit(all_predictors(), all_outcomes())
```

```
simpson_wf <- workflow() %>%
  add_recipe(simpson_recipe) %>%
  add_model(rf_spec)
```

Tune

```
simpson_res <- tune_grid(
  simpson_wf,
  resamples = folds,
  grid       = rf_grid,
  metrics    = metric_set(rmse, rsq, mae)
)

autoplot(simpson_res)
```



```
show_best(simpson_res, metric = "rmse", n = 5) %>%
  select(mtry, min_n, mean, std_err) %>%
  mutate(across(c(mean, std_err), ~ round(.x, 3))) %>%
  rename(
    "Mean RMSE" = mean,
    "Std. error" = std_err
  ) %>%
```

```
knitr::kable(caption = "Top 5 hyperparameter combinations:
  Simpson diversity (ranked by RMSE)")
```

Table 4: Top 5 hyperparameter combinations: Simpson diversity (ranked by RMSE)

mtry	min_n	Mean RMSE	Std. error
6	15	0.142	0.011
5	15	0.142	0.012
8	15	0.143	0.011
6	11	0.144	0.012
5	11	0.145	0.012

Final Model

```
best_simpson <- select_best(simpson_res, metric = "rmse")
final_simpson_wf <- finalize_workflow(simpson_wf, best_simpson)
simpson_fit <- fit(final_simpson_wf, data = df)

simpson_engine_fit <- simpson_fit %>% extract_fit_engine()

tibble::tibble(
  Parameter = c("mtry", "min.node.size", "num.trees",
    "OOB prediction error (MSE)", "R2 (OOB)"),
  Value = c(
    simpson_engine_fit$mtry,
    simpson_engine_fit$min.node.size,
    simpson_engine_fit$num.trees,
    round(simpson_engine_fit$prediction.error, 3),
    round(simpson_engine_fit$r.squared, 3)
  )
) %>%
  knitr::kable(caption = "Final random forest model: Simpson diversity")
```

Table 5: Final random forest model: Simpson diversity

Parameter	Value
mtry	6.000
min.node.size	15.000
num.trees	500.000
OOB prediction error (MSE)	0.020
R ² (OOB)	0.375

Cross-Validation Performance


```

simpson_cv_metrics <- collect_metrics(simpson_res, summarize = FALSE) %>%
  filter(mtry == best_simpson$mtry, min_n == best_simpson$min_n) %>%
  left_join(fold_city_lookup, by = "id")

simpson_cv_metrics %>%
  select(city, .metric, .estimate) %>%
  tidyr::pivot_wider(names_from = .metric, values_from = .estimate) %>%
  mutate(across(where(is.numeric), ~ round(.x, 3))) %>%
  rename(
    "City (held out)" = city,
    "RMSE" = rmse,
    "R2" = rsq,
    "MAE" = mae
  ) %>%
  knitr::kable(caption = "Cross-validated performance by held-out city:
    Simpson diversity")

```

Table 6: Cross-validated performance by held-out city: Simpson diversity

City (held out)	RMSE	R ²	MAE
Nottingham	0.128	0.564	0.079
Exeter	0.127	0.363	0.081
Newcastle	0.165	0.265	0.109
Bristol	0.118	0.541	0.066
Gateshead	0.173	0.586	0.099

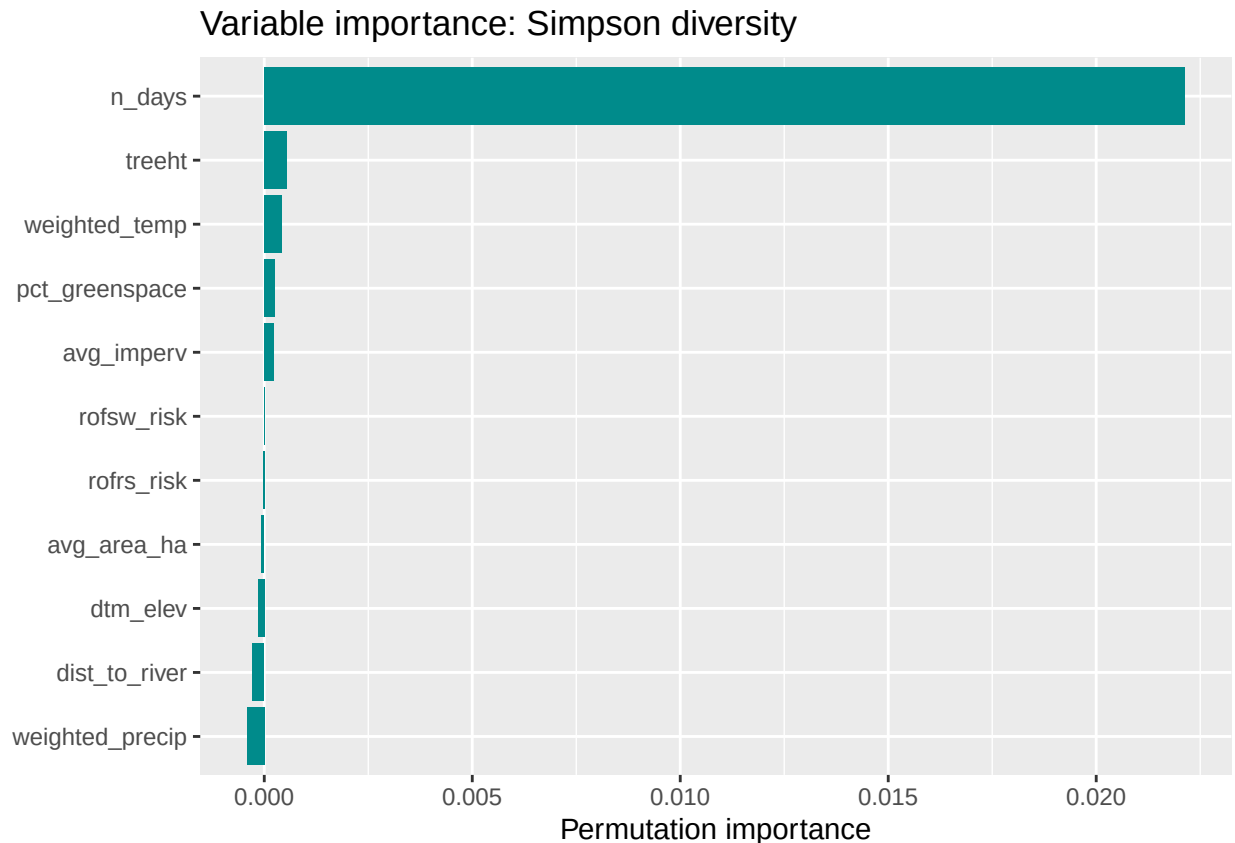
Variable Importance

```

p2 <- simpson_fit %>%
  extract_fit_engine() %>%
  ranger::importance() %>%
  sort(decreasing = TRUE) %>%
  as.data.frame() %>%
  tibble::rownames_to_column("variable") %>%
  rename(importance = 2) %>%
  ggplot(aes(x = reorder(variable, importance), y = importance)) +
  geom_col(fill = "cyan4") +
  coord_flip() +
  labs(title = "Variable importance: Simpson diversity",
    x = NULL,
    y = "Permutation importance")

```

p2



```
#ggsave("VI_RF_Reg_Simp.png", p2, width = 12, height = 5, dpi = 300)
```

This models shows an $R^2 = 0.375$ and city based cross-validation R^2 scores ranging from 0.265 to 0.586. Similar to the richness RF model, **n_days** is the dominant predictor. There are several predictors with negative permutation importance (**weighted_precip**, **dist_to_river** etc.) which happens when the model performance improves with shuffled data compared to the original data. It is possible that this is due to the small sample size, non-predictive/noisy features, or correlated features within the dataset.

Exploring **n_days** as a variable

Variable importance in both RF models indicate that **n_days** greatly dominates over all other environmental variables. This analysis looks at **n_days** as a predictor.

Correlation with other predictors

```
numeric_predictors <- df %>%
  select(n_days, all_of(predictors)) %>%
  select(where(is.numeric)) %>%
  names()

df %>%
  select(all_of(numeric_predictors)) %>%
```

```
cor(use = "pairwise.complete.obs") %>%
as.data.frame() %>%
select(n_days) %>%
arrange(desc(abs(n_days)))
```

```
##              n_days
## n_days      1.00000000
## pct_greenspace 0.21675973
## weighted_precip 0.18773822
## weighted_temp  0.17863501
## dist_to_river  -0.10707961
## dtm_elev       -0.10282919
## avg_imperv     0.09945388
## treeht         0.07285328
## avg_area_ha    0.04021645
```

`n_days` shows only weak linear correlations with the other predictors, largest correlation being 0.22. This suggests that `n_days` is not strongly linearly associated with any individual predictor and is, therefore, unlikely to be capturing the same linear information as other variables. However, this doesn't conclude that statistical significance is independent of the other predictors since more complex, non-linear relationships may exist.

Scatter correlation

```
# Dataset containing only numeric predictors
cor_data <- df %>%
  select(all_of(numeric_predictors))

# Plot a scatter correlation matrix
cor_plot <- GGally::ggpairs(cor_data,
  lower = list(continuous = GGally::wrap(
    "points", alpha = 0.4, size = 0.8)),
  upper = list(continuous = GGally::wrap("cor", size = 3)),
  diag = list(continuous = GGally::wrap("densityDiag")))
cor_plot
```

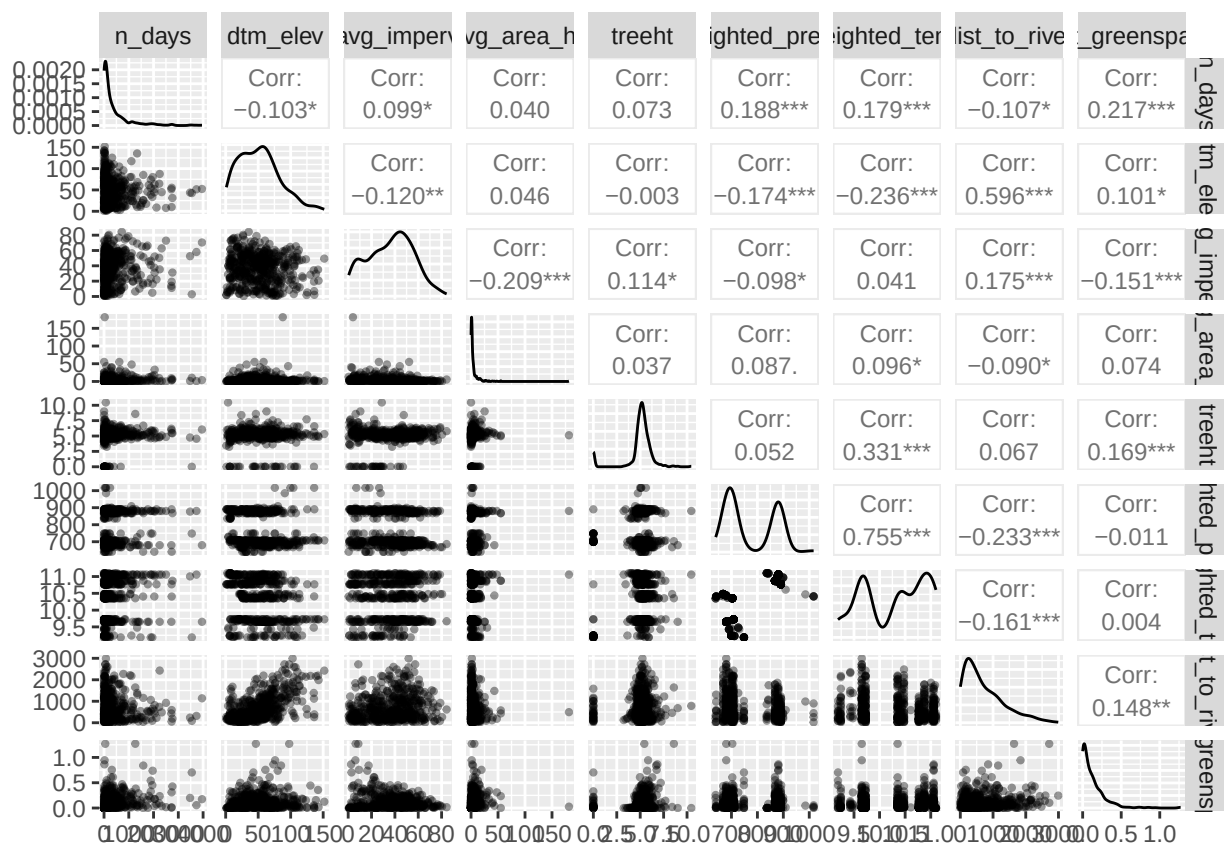
```
## Warning: Removed 4 rows containing missing values
```

```
## Warning: Removed 4 rows containing missing values or values outside the scale range
## ('geom_point()').
```

```
## Warning: Removed 4 rows containing non-finite outside the scale range
## ('stat_density()').
```

```
## Warning: Removed 4 rows containing missing values
## Removed 4 rows containing missing values
## Removed 4 rows containing missing values
## Removed 4 rows containing missing values
## Removed 4 rows containing missing values
## Removed 4 rows containing missing values
## Removed 4 rows containing missing values
```

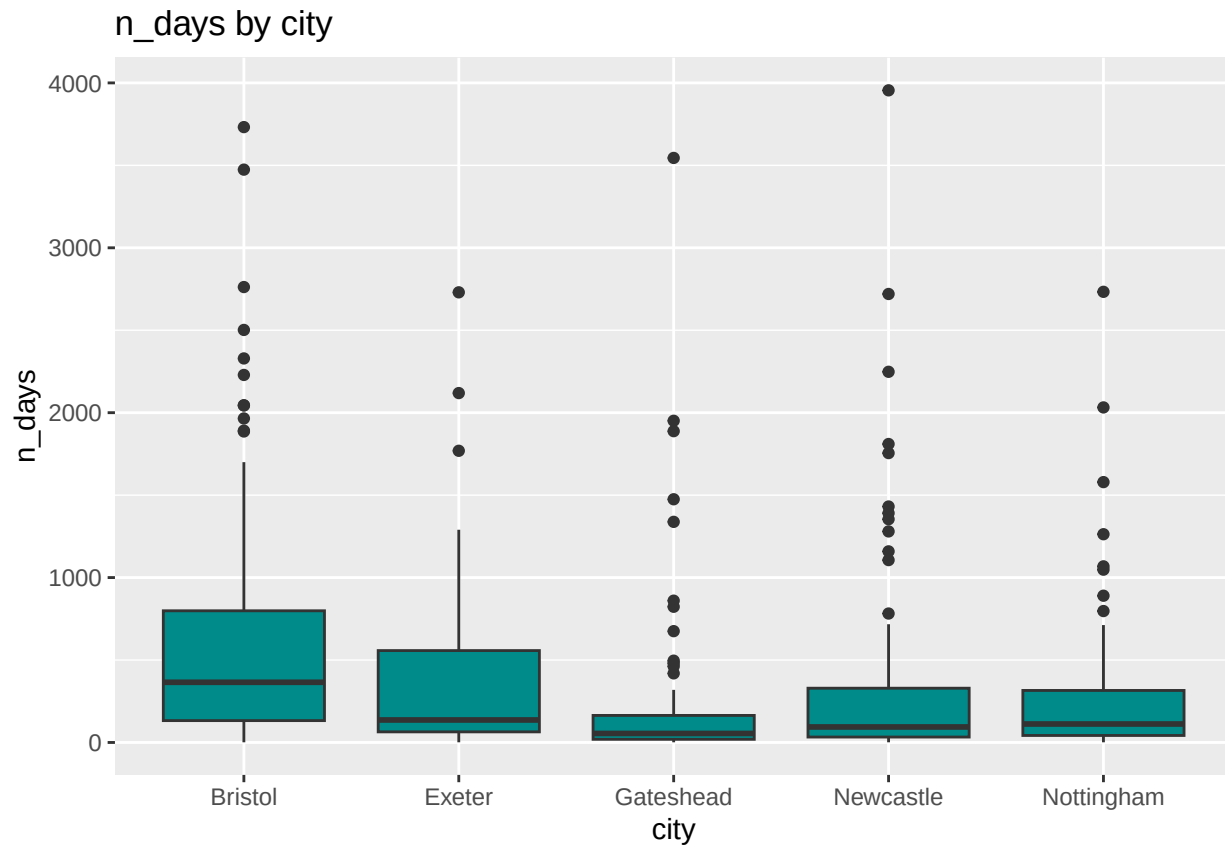
```
## Warning: Removed 4 rows containing missing values or values outside the scale range
## ('geom_point()').
## Removed 4 rows containing missing values or values outside the scale range
## ('geom_point()').
## Removed 4 rows containing missing values or values outside the scale range
## ('geom_point()').
## Removed 4 rows containing missing values or values outside the scale range
## ('geom_point()').
## Removed 4 rows containing missing values or values outside the scale range
## ('geom_point()').
## Removed 4 rows containing missing values or values outside the scale range
## ('geom_point()').
## Removed 4 rows containing missing values or values outside the scale range
## ('geom_point()').
```



This plot shows `n_days` is highly skewed, with most observations concentrated at relatively low values and a small number of extreme large values. Pearson's correlation coefficient is sensitive to skewed distributions and extreme observations, meaning this may not fully characterise the relationships involving `n_days`.

Variation by city

```
p3 <- ggplot(df, aes(x = city, y = n_days)) +
  geom_boxplot(fill = "cyan4") +
  labs(title = "n_days by city")
p3
```



```
#ggsave("n_days_by_city.png", p3, width = 12, height = 5, dpi = 300)
```

```
city_aov <- aov(n_days ~ city, data = df)
summary(city_aov)
```

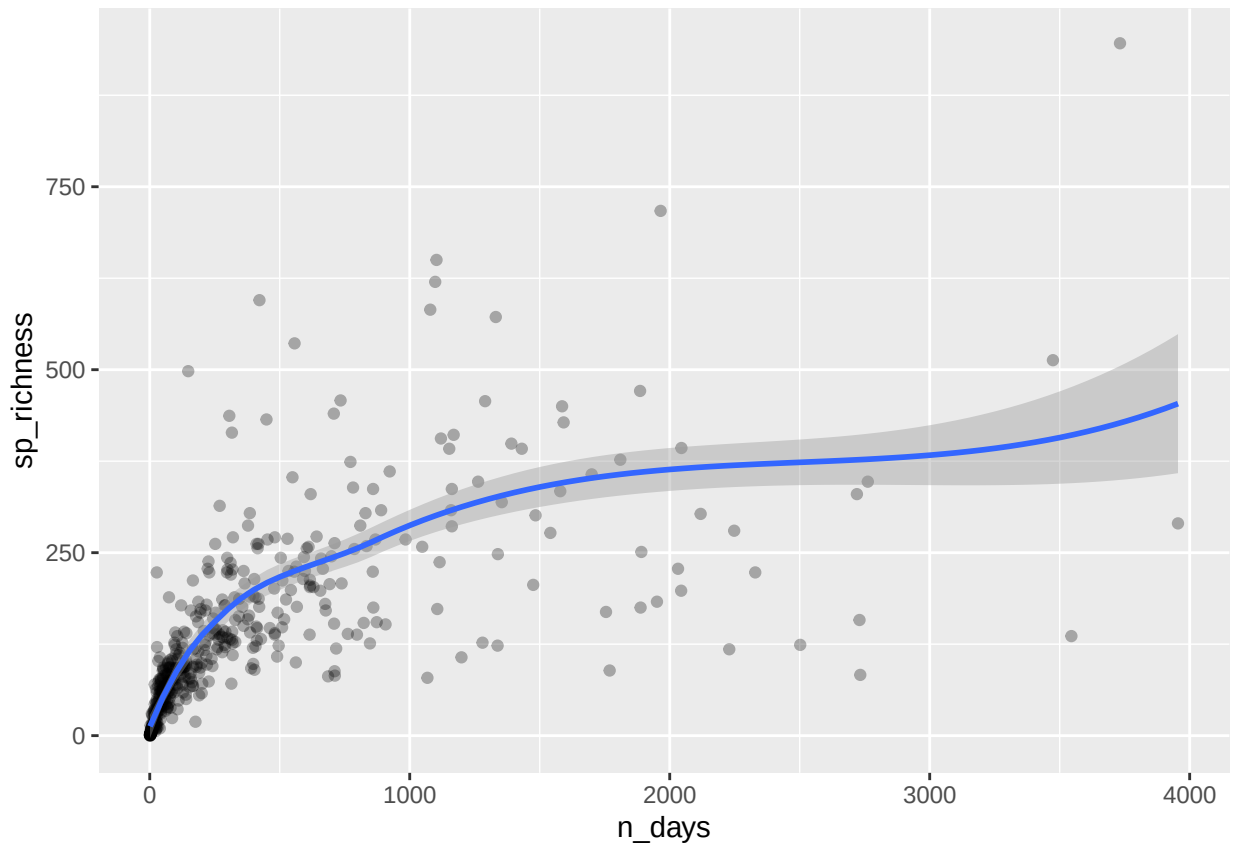
```
##              Df    Sum Sq Mean Sq F value    Pr(>F)
## city          4  9582454 2395614    7.136 1.38e-05 ***
## Residuals    484 162490904  335725
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
ss <- summary(city_aov)[[1]][["Sum Sq"]]
r2_city <- ss[1] / sum(ss)
r2_city
```

```
## [1] 0.05568819
```

```
p4 <- df %>%
  select(n_days, sp_richness) %>%
  ggplot(aes(x = n_days, y = sp_richness)) +
  geom_point(alpha = 0.3) +
  geom_smooth()
p4
```

```
## 'geom_smooth()' using method = 'loess' and formula = 'y ~ x'
```



```
#ggsave("ndays_sp_rich.png", p4, width = 12, height = 5, dpi = 300)
```

There is a statistically significant difference in **n_days** among cities (ANOVA, $p = 1.38 \times 10^{-5}$). City accounted for approximately 5.6% of the variation in **n_days** ($R^2 = 0.056$), indicating that although sampling effort differs significantly among cities, the effect size is relatively small.

The **sp_richness/n_days** curve suggests richness is associated with number of sampling days. It shows a steep increase in species detection between 0-500 days, with most of the samples clustered in this area. After 1000 days, it begins to plateau although samples become sparse here so there is increased uncertainty about the relationship at these values. Scatter is consistently wide throughout the graph.

RF Model without n_days

The model is rerun, as before, without **n_days** for comparison.

```
predictors_nodays <- c("dtm_elev", "avg_imperv", "rofsw_risk", "rofrs_risk",
  "avg_area_ha", "treeht", "weighted_precip", "weighted_temp",
  "dist_to_river", "pct_greenspace")

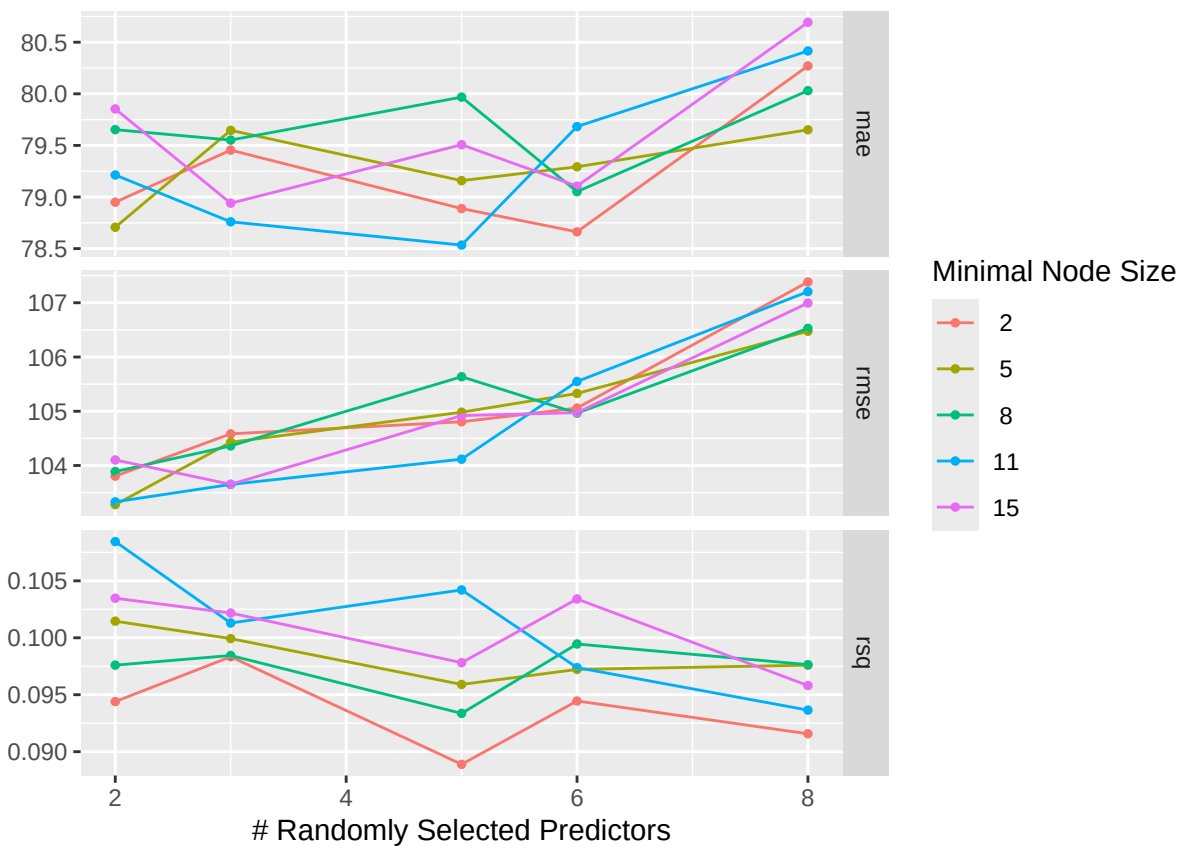
richness_recipe_nd <- recipe(
  as.formula(paste("sp_richness ~", paste(predictors_nodays, collapse = " + "))),
  data = df) %>%
  step_naomit(all_predictors(), all_outcomes())
```

```
richness_wf_nd <- workflow() %>%
  add_recipe(richness_recipe_nd) %>%
  add_model(rf_spec)

rf_grid <- grid_regular(
  mtry(range = c(2, 8)),
  min_n(range = c(2, 15)),
  levels = 5) #Total of 25 combinations in the grid.

richness_res_nd <- tune_grid(
  richness_wf_nd,
  resamples = folds,
  grid      = rf_grid,
  metrics   = metric_set(rmse, rsq, mae))

autoplot(richness_res_nd)
```



```
show_best(richness_res_nd, metric = "rmse", n = 5) %>%
  select(mtry, min_n, mean, std_err) %>%
  mutate(across(c(mean, std_err), ~ round(.x, 3))) %>%
  rename(
    "mtry" = mtry,
    "min_n" = min_n,
    "Mean RMSE" = mean,
    "Std. error" = std_err)
```

```
) %>%
knitr::kable(caption = "Top 5 hyperparameter combinations:
species richness (ranked by RMSE) no `n_days`")
```

Table 7: Top 5 hyperparameter combinations: species richness (ranked by RMSE) no `n_days`

mtry	min_n	Mean RMSE	Std. error
2	5	103.277	20.955
2	11	103.330	20.361
3	11	103.651	20.507
3	15	103.654	20.497
2	2	103.804	20.677

```
best_richness_nd <- select_best(richness_res_nd, metric = "rmse")
final_richness_wf_nd <- finalize_workflow(richness_wf_nd, best_richness_nd)
richness_fit_nd <- fit(final_richness_wf_nd, data = df)

richness_engine_fit_nd <- richness_fit_nd %>% extract_fit_engine()

tibble::tibble(
  Parameter = c("mtry", "min.node.size", "num.trees",
                "OOB prediction error (MSE)", "R2 (OOB)"),
  Value = c(
    richness_engine_fit_nd$mtry,
    richness_engine_fit_nd$min.node.size,
    richness_engine_fit_nd$num.trees,
    round(richness_engine_fit_nd$prediction.error, 3),
    round(richness_engine_fit_nd$r.squared, 3)
  )
) %>%
knitr::kable(
  caption = "Final random forest model: species richness no `n_days`")
```

Table 8: Final random forest model: species richness no `n_days`

Parameter	Value
mtry	2.000
min.node.size	5.000
num.trees	500.000
OOB prediction error (MSE)	12724.973
R ² (OOB)	0.302

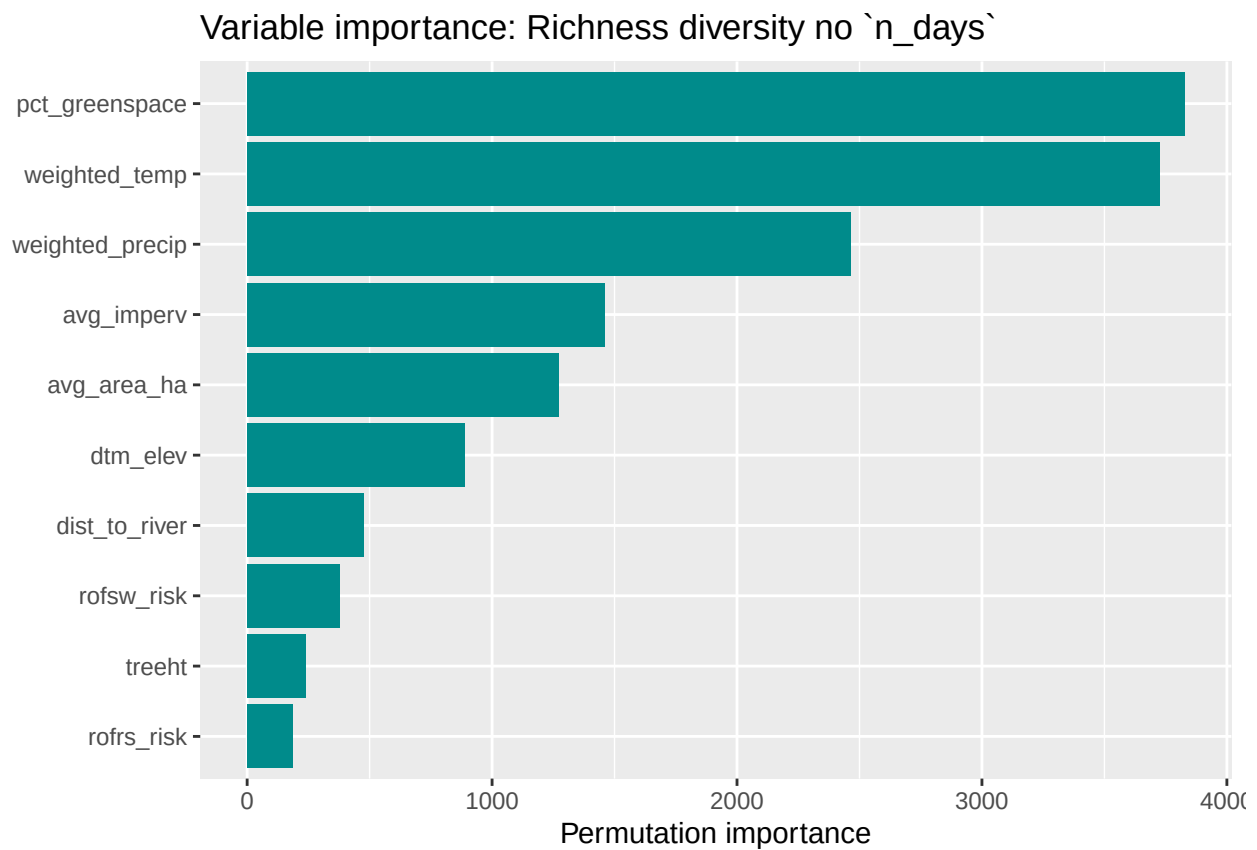
The R^2 (0.302) is considerably lower than the original richness RF model's R^2 (0.606).

```
p5 <- richness_fit_nd %>%
  extract_fit_engine() %>%
  ranger::importance() %>%
  sort(decreasing = TRUE) %>%
```



```
as.data.frame() %>%
  tibble::rownames_to_column("variable") %>%
  rename(importance = 2) %>%
  ggplot(aes(x = reorder(variable, importance), y = importance)) +
  geom_col(fill = "cyan4") +
  coord_flip() +
  labs(title = "Variable importance: Richness diversity no `n_days`",
       x = NULL,
       y = "Permutation importance")
```

p5



```
#ggsave("VI_RF_Reg_ND_Rich.png", p5, width = 12, height = 5, dpi = 300)
```

For comparison with published literature, the next Rmd looks at Random Forest using classification and ROC AUC.